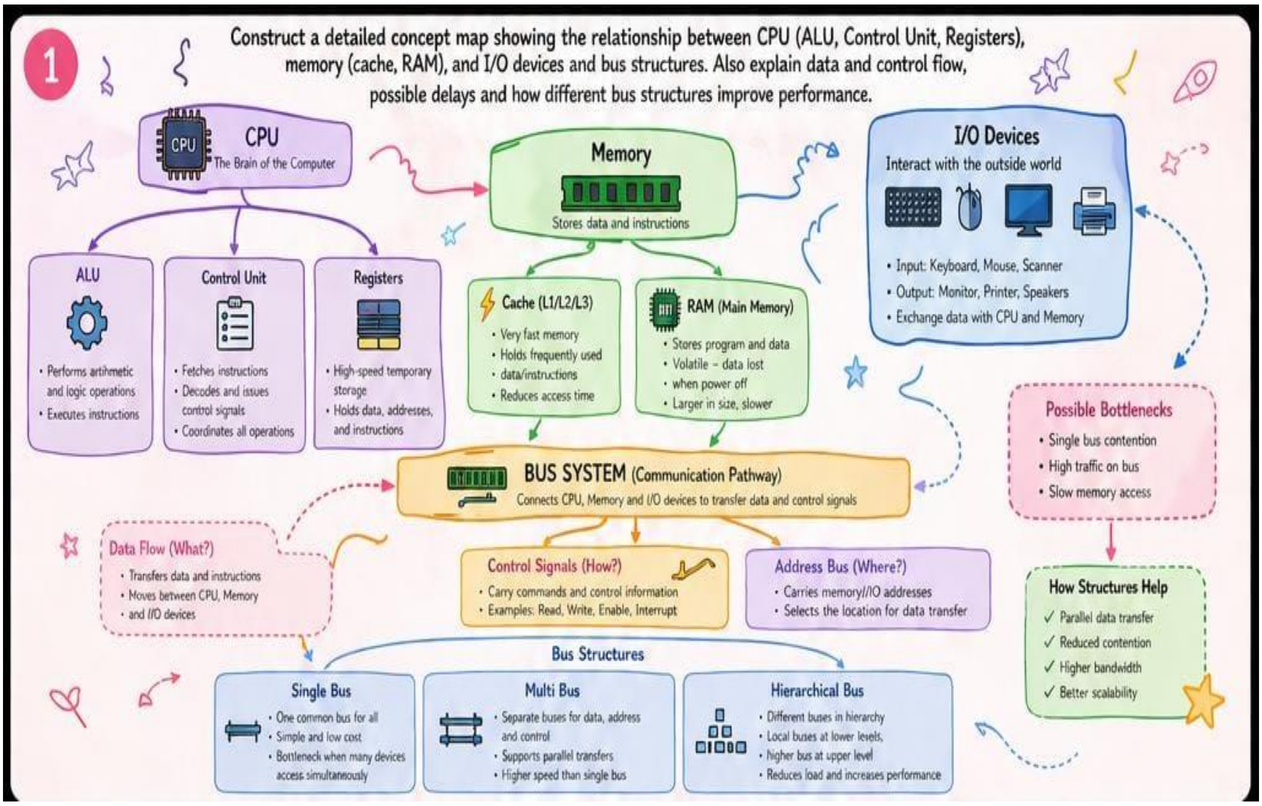
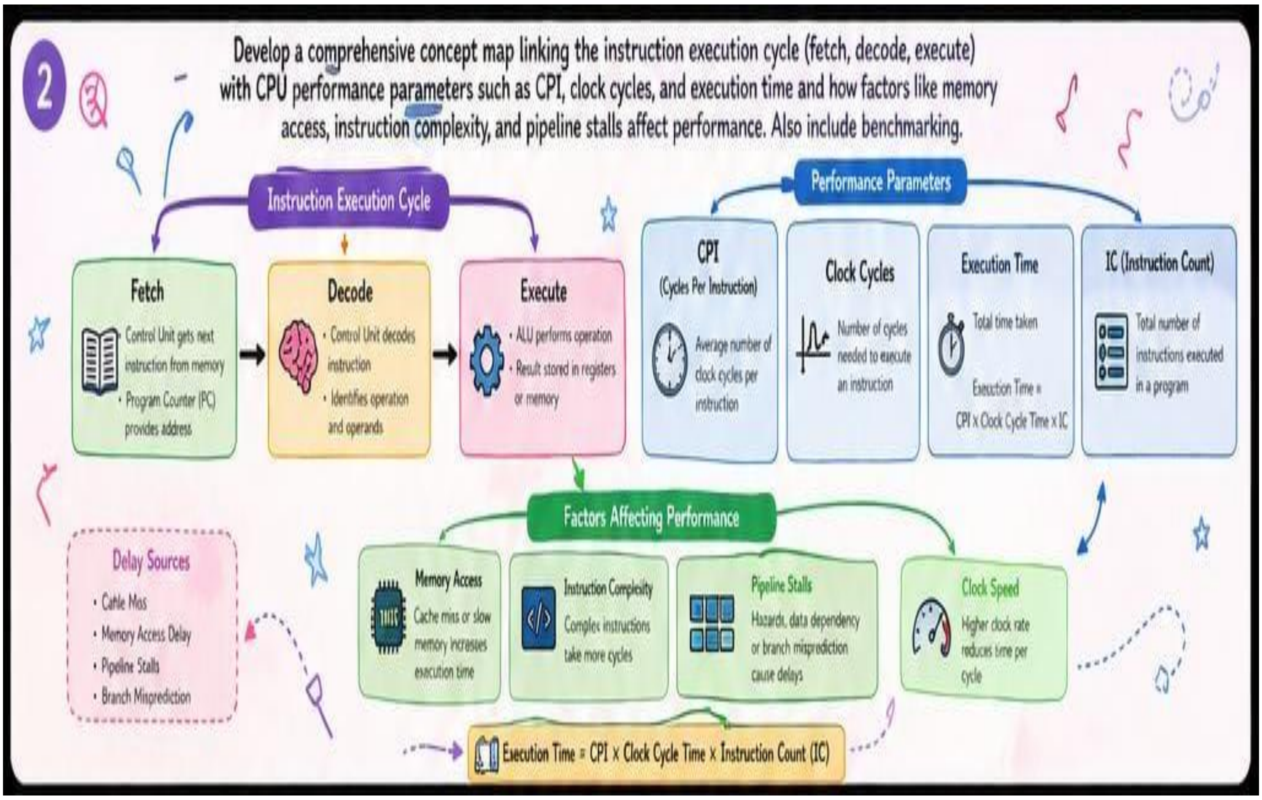
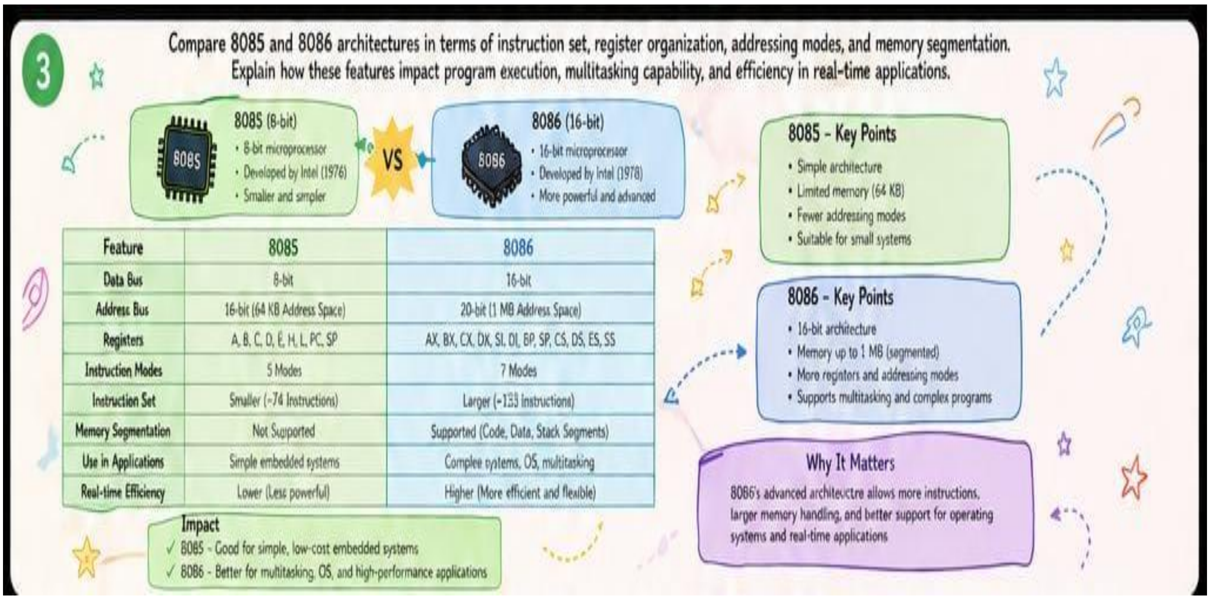
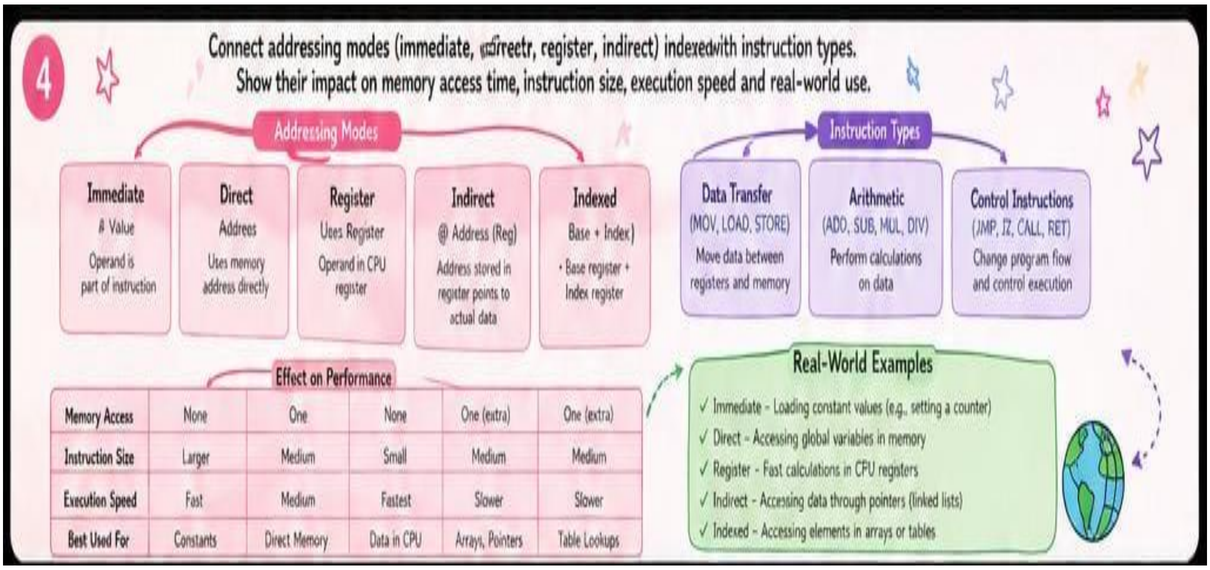

1
Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices and bus structures. Also explain data and control flow, possible delays and how different bus structures improve performance.
CPU
The Brain of the Computer
ALU
• Performs arithmetic and logic operations
• Executes instructions
Control Unit
• Fetches instructions
• Decodes and issues control signals
• Coordinates all operations
Registers
• High-speed temporary storage
• Holds data, addresses, and instructions
Memory
Stores data and instructions
Cache (L1/L2/L3)
• Very fast memory
• Holds frequently used data/instructions
• Reduces access time
RAM (Main Memory)
• Stores program and data
• Volatile – data lost when power off
• Larger in size, slower
I/O Devices
Interact with the outside world
• Input: Keyboard, Mouse, Scanner
• Output: Monitor, Printer, Speakers
• Exchange data with CPU and Memory
Possible Bottlenecks
• Single bus contention
• High traffic on bus
• Slow memory access
BUS SYSTEM (Communication Pathway)
Connects CPU, Memory and I/O devices to transfer data and control signals
Data Flow (What?)
• Transfers data and instructions
• Moves between CPU, Memory and I/O devices
Control Signals (How?)
• Carry commands and control information
• Examples: Read, Write, Enable, Interrupt
Address Bus (Where?)
• Carries memory/I/O addresses
• Selects the location for data transfer
How Structures Help
✓ Parallel data transfer
✓ Reduced contention
✓ Higher bandwidth
✓ Better scalability
Bus Structures
Single Bus
• One common bus for all
• Simple and low cost
• Bottleneck when many devices access simultaneously
Multi Bus
• Separate buses for data, address and control
• Supports parallel transfers
• Higher speed than single bus
Hierarchical Bus
• Different buses in hierarchy
• Local buses at lower levels, higher bus at upper level
• Reduces load and increases performance


2
Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time and how factors like memory access, instruction complexity, and pipeline stalls affect performance. Also include benchmarking.
Instruction Execution Cycle
Fetch
• Control Unit gets next instruction from memory
• Program Counter (PC) provides address
Decode
• Control Unit decodes instruction
• Identifies operation and operands
Execute
• ALU performs operation
• Result stored in registers or memory
Performance Parameters
CPI
(Cycles Per Instruction)
Average number of clock cycles per instruction
Clock Cycles
Number of cycles needed to execute an instruction
Execution Time
Total time taken
Execution Time = CPI × Clock Cycle Time × IC
IC (Instruction Count)
Total number of instructions executed in a program
Factors Affecting Performance
Delay Sources
• Cache Miss
• Memory Access Delay
• Pipeline Stalls
• Branch Misprediction
Memory Access
Cache miss or slow memory increases execution time
Instruction Complexity
Complex instructions take more cycles
Pipeline Stalls
Hazards, data dependency or branch misprediction cause delays
Clock Speed
Higher clock rate reduces time per cycle
Execution Time = CPI × Clock Cycle Time × Instruction Count (IC)


3
Compare 8085 and 8086 architectures in terms of instruction set, register organization, addressing modes, and memory segmentation.
Explain how these features impact program execution, multitasking capability, and efficiency in real-time applications.
8085
8085 (8-bit)
• 8-bit microprocessor
• Developed by Intel (1976)
• Smaller and simpler
VS
8086
8086 (16-bit)
• 16-bit microprocessor
• Developed by Intel (1978)
• More powerful and advanced

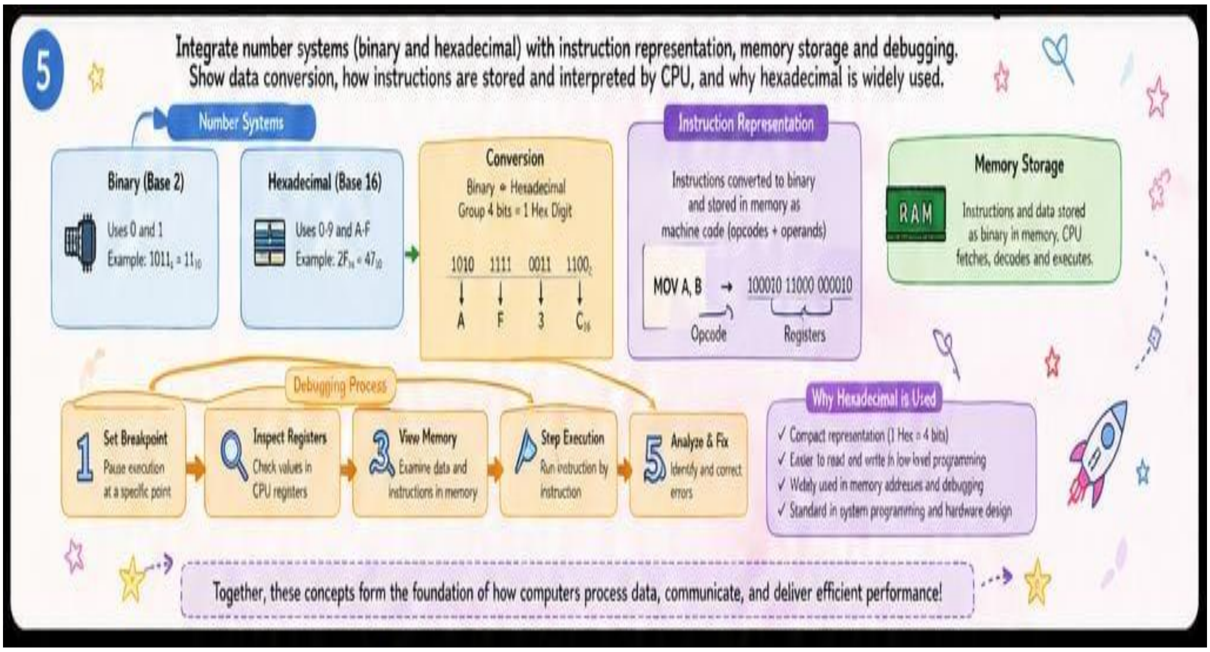
Feature	8085	8086
Data Bus	8-bit	16-bit
Address Bus	16-bit (64 KB Address Space)	20-bit (1 MB Address Space)
Registers	A, B, C, D, E, H, L, PC, SP	AX, BX, CX, DX, SI, DI, BP, SP, CS, DS, ES, SS
Instruction Modes	5 Modes	7 Modes
Instruction Set	Smaller (~74 Instructions)	Larger (~133 Instructions)
Memory Segmentation	Not Supported	Supported (Code, Data, Stack Segments)
Use in Applications	Simple embedded systems	Complex systems, OS, multitasking
Real-time Efficiency	Lower (Less powerful)	Higher (More efficient and flexible)

Impact
✓ 8085 - Good for simple, low-cost embedded systems
✓ 8086 - Better for multitasking, OS, and high-performance applications
8085 - Key Points
• Simple architecture
• Limited memory (64 KB)
• Fewer addressing modes
• Suitable for small systems
8086 - Key Points
• 16-bit architecture
• Memory up to 1 MB (segmented)
• More registers and addressing modes
• Supports multitasking and complex programs
Why It Matters
8086's advanced architeuctre allows more instructions, larger memory handling, and better support for operating systems and real-time applications


4
Connect addressing modes (immediate, direetr, register, indirect) indexedwith instruction types.
Show their impact on memory access time, instruction size, execution speed and real-world use.
Addressing Modes
Immediate
Value
Operand is part of instruction
Direct
Address
Uses memory address directly
Register
Uses Register
Operand in CPU register
Indirect
@ Address (Reg)
Address stored in register points to actual data
Indexed
Base + Index
• Base register + Index register
Instruction Types
Data Transfer
(MOV, LOAD, STORE)
Move data between registers and memory
Arithmetic
(ADD, SUB, MUL, DIV)
Perform calculations on data
Control Instructions
(JMP, JZ, CALL, RET)
Change program flow and control execution
Effect on Performance

Memory Access	None	One	None	One (extra)	One (extra)
Instruction Size	Larger	Medium	Small	Medium	Medium
Execution Speed	Fast	Medium	Fastest	Slower	Slower
Best Used For	Constants	Direct Memory	Data in CPU	Arrays, Pointers	Table Lookups

Real-World Examples
✓ Immediate - Loading constant values (e.g., setting a counter)
✓ Direct - Accessing global variables in memory
✓ Register - Fast calculations in CPU registers
✓ Indirect - Accessing data through pointers (linked lists)
✓ Indexed - Accessing elements in arrays or tables


5
Integrate number systems (binary and hexadecimal) with instruction representation, memory storage and debugging.
Show data conversion, how instructions are stored and interpreted by CPU, and why hexadecimal is widely used.
Number Systems
Binary (Base 2)
Uses 0 and 1
Example: $1011_2 = 11_{10}$
Hexadecimal (Base 16)
Uses 0-9 and A-F
Example: $2F_{16} = 47_{10}$
Conversion
Binary ↔ Hexadecimal
Group 4 bits = 1 Hex Digit
1010 1111 0011 1100_2
A F 3 C_{16}
Instruction Representation
Instructions converted to binary and stored in memory as machine code (opcodes + operands)
MOV A, B → 100010 11000 000010
Opcode
Registers
Memory Storage
RAM
Instructions and data stored as binary in memory, CPU fetches, decodes and executes.
Debugging Process
1 Set Breakpoint
Pause execution at a specific point
Inspect Registers
Check values in CPU registers
3 View Memory
Examine data and instructions in memory
Step Execution
Run instruction by instruction
5 Analyze & Fix
Identify and correct errors
Why Hexadecimal is Used
✓ Compact representation (1 Hex = 4 bits)
✓ Easier to read and write in low level programming
✓ Widely used in memory addresses and debugging
✓ Standard in system programming and hardware design
Together, these concepts form the foundation of how computers process data, communicate, and deliver efficient performance!